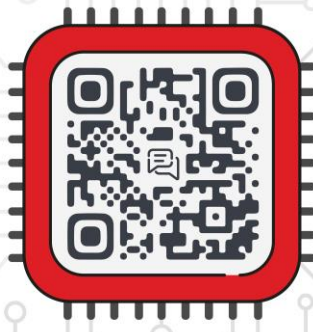




7



5



7.5 sp

TD

كلية الهندسة المعلوماتية

السنة الأولى

التعددية (Polymorphism)

د. ماهر صارم



RB Informatix; 21/5/2026

محتوى مجاني غير مخصص للبيع التجاري

البرمجة 2

التَّعدُّدية (Polymorphism)

قد قمنا بالشرح سابقاً عن التعددية في مثال سابق ورد فيه تطبيق لفكرة التحميل الزائد (Overloading) الذي يتمثل في كتابة توابع بنفس الاسم ولكن باختلاف في عدد الوسائط (parameters) أو نوعها وتكون هذه التوابع موجودة في ذات الكلاس.

يوجد نوع آخر من أنواع التعددية وهو ال Override ويظهر هذا المفهوم حصراً بين كلاسيتين مختلفتين (كلاس أب وكلاس ابن).

ما هو ال Override؟

هو ميزة تسمح لكلاس الابن أن يعيد تعريف أو تغيير طريقة عمل تابع ما موجود مسبقاً لدى الأب بما يتناسب مع احتياجات كلاس الابن الخاصة.

شرط استعمال ال Override:



أن يكون تابع كلاس الابن لديه نفس اسم ووسائط تابع كلاس الأب تماماً ولذلك يُفضل دوماً كتابة العبارة التوضيحية (@Override) قبل القيام بعملية ال Override لأن ذلك سيساعدك بالتأكد من أنك تقوم بإعادة الكتابة بشكل صحيح إذ سيخبرك ال compiler بوجود خطأ في حال أخطأت باسم التابع مثلاً.

مقارنة توضيحية بين ال Override وال Overload:

Overloading	Overriding
كتابة أكثر من تابع بنفس الاسم ولكن بمدخلات مختلفة.	إعادة تعريف نفس التابع بنفس المدخلات في كلاس ابن لتغيير سلوكه.
يحدث داخل نفس الكلاس.	يحدث بين كلاسين بينهما علاقة وراثة.
يتم تحديد الدالة التي ستنفذ أثناء وقت الترجمة (Compile-time).	يتم تحديد الدالة التي ستنفذ في وقت التشغيل (Runtime).

توضيح لسبب الاختلاف في وقت تحديد الدالة التي ستنفذ:

- في ال Overload تكون جميع المعلومات اللازمة لاختيار التابع متوفرة أثناء الترجمة، مثل عدد ال parameters أو نوعها، لذلك يحدد المترجم أي تابع سيتم تنفيذه.
- في ال Override يعرف المترجم أن التابع موجود في نوع المرجع، لكنه لا يحدد النسخة النهائية التي ستنفذ إلا أثناء وقت التشغيل، وذلك حسب نوع الكائن الفعلي الموجود في الذاكرة.

ال Upcasting:

هو إسناد كائن من الكلاس الابن إلى مرجع من نوع الكلاس الأب، بحيث يبقى الكائن الفعلي من نوع الابن، لكن التعامل معه يتم من خلال مرجع الأب؛ لأننا لا نغير الكائن نفسه، بل نغير نوع المرجع الذي يشير إليه.

مثال توضيحي لشرح المفاهيم السابقة:

```
public class A {
    int x = 10;
    void print () {
        System.out.println ("A = " + x);
    }
}
```

A كلاس أب و B كلاس ابن

الشرح:

```
public class B extends A{
    int x = 20;
    @Override
    void print () {
        System.out.println ("B = " + x);
    }
}
```

```
1- public class Main {
2-     public static void main (String[] args)
3-     {
4-         A b = new B ();
5-         System.out.println (b.x);
6-         b.print ();
7-     }
8- }
```

قمنا بتعريف كلاس

في السطر الثالث من الـ main قمنا بعملية Upcasting.

في السطر الرابع من الـ main سيعود لنوع المرجع أثناء الترجمة وليس للنوع الفعلي للكائن b ولذلك ستطبع الـ x الموجودة في A.

في السطر الخامس من الـ main سيعود للنوع الفعلي للكائن b أثناء التشغيل أي للكلاس نفسه الذي استدعي منه الباني ولذلك ستطبع الـ x الموجودة في B.

الخلاصة: المتغيرات تحدّد حسب نوع المرجع، أما التوابع التي تم عمل Override لها فتحدّد حسب نوع الكائن الفعلي أثناء وقت التشغيل.

ملاحظة:

في المثال السابق لو قمنا بكتابة تابع يطبع (Hello) في الابن B ولم نقم بتعريفه في الأب A وحاولنا استدعاء هذا التابع عن طريق b سيظهر Error لأنّ المرجع بعد الـ Upcasting يسمح بالوصول فقط إلى الأعضاء المعرفة في نوع الأب، أما التوابع الموجودة فقط في الابن فلا يمكن استدعاؤها من خلال هذا المرجع.

مثال واقعي:

```
public class Customer {
    private int acc;
    private String name;
    protected double balance;
    Customer(int acc, String name) {
        balance = 0;
        this.acc = acc;
        this.name = name; }
    void deposit(double balance) {}
    void withdraw(double balance) {}
    void Rate() {} }
```



```
public class Current extends Customer{
    private double rate;
    Current(int acc, String name) {
        super(acc, name);
        rate = 0.05;
    }
    @Override
    void deposit(double balance) {
        this.balance += balance;
        System.out.println("deposit to Current");
    }
    @Override
    void withdraw(double balance) {
        this.balance -= balance;
        System.out.println("withdraw Current");
    }
    @Override
    void Rate() {
        balance = balance + (balance * rate);
        System.out.println("Rate To Current");}
}
```

```
public class Saving extends Customer{
    private double rate;
    Saving(int acc, String name) {
        super(acc, name);
        rate = 0.09;}
    @Override
    void deposit(double balance) {
        this.balance += balance;
        System.out.println("deposit to Saving");}
    @Override
    void withdraw(double balance) {
        this.balance -= balance;
        System.out.println("withdraw from Saving");
    }
    @Override
    void Rate() {
        balance = balance + (balance * rate); }
}
```

```
import java.util.ArrayList;
import java.util.List;
public class Main {
    public static void main (String[] args)
    {
        Customer c1 = new Current(111, "Majd");
        Customer c2 = new Saving(222, "Ahmad");

        List<Customer> C = new ArrayList<>();

        C.add(c1);
        C.add(c2);

        for (Customer cust : C) {
            cust.Rate();
        }
    }
}
```

شرح أهم الأفكار في المثال:

- فكرة الـ override:

عرفنا 3 توابع فارغة في

الكلاس الأب Customer

وفي كل من

الكلاسين Current و Saving

قمنا بعملية override للتوابع

الثلاث بما يتناسب مع الكلاس

الموجودة فيه.

- فكرة الـ Upcasting:

في الـ main قمنا بإنشاء كائنين c1 و c2 من مرجع الأب ولكن باستعمال بانيتين مختلفتين، والهدف من هذه العملية هو إنشاء الـ ArrayList من النوع Customer وإضافتهم إليها بالرغم من اختلافهم بالخصائص.

نلاحظ كذلك استعمال حلقة for تدعى for-each أو enhanced for loop (أي من أجل كل كائن من نوع Customer في المصفوفة C) وبذلك سيمر على جميع العناصر في C وينفذ تعليمة (cust.Rate();) عليهم جميعاً.

انتهت المحاضرة السابعة 🎓